

Asset Slot Demotion, Deploy Runbook

✓ COMPLETE, 2026-05-29

The full A-E migration has shipped on **both dev and prod**. All five columns are dropped, `__slots` is backfilled + verified, both migration ledgers are consistent, and the Phase E fallbacks are removed (`__slots` is the sole read path). - Dev: columns dropped in the 2026-05-28 rehearsal; ledger repaired 2026-05-29. - Prod: code deployed via `dev→main→prod` (`6c9d19c`); Tier 1/2 run via `db query --linked` (backfill → verify 0-missing → CONCURRENTLY indexes → drop); ledger recorded. 213/218 rows carry `__slots`. - Phase E dead-code removal: commit `77d3618` → prod `b50a760`.

The procedure below is retained as the validated reference (e.g. for the future **Work** slot demotion, which follows the same pattern). It is no longer a pending action.

Status: Authoritative ordered procedure. Supersedes the scattered guidance in migration-file comments, `project_asset_slot_demotion` memory, and `project_todo_slot_demotion_phase_e` memory.

What this is: Demoting five columns (`dev_name`, `priority`, `item_type`, `team`, `status`) off `replicated_assets` into `meta["__slots"]`. Phases A-D (code + migration files) were authored 2026-05-28; this runbook sequenced the safe landing, executed 2026-05-29.

0. The two things the old plan got wrong (read first)

The Phase E memory says the only reader to fix is `routes/assets.py`. **That is incomplete.** Audit on 2026-05-28 found two readers that break *harder* than the `assets.py` fallback, they put dropped columns into PostgREST `select` /filter params, which returns **HTTP 400** the instant the column is gone. These are not Phase E cleanup; they **gate the column drop** and must ship in the same deploy as the Phase A-D code, before any migration runs.

Reader	Line	Break	Severity
<code>lib/scheduler.py</code> <code>build_schedule()</code>	<code>select:</code> <code>"name,item_type,product,priority,project_date,meta,canonical_asset_id"</code> (~L108); reads <code>asset_row.get("item_type")</code> (~L138), <code>asset_row.get("priority")</code> (~L153), <code>asset_row.get(slot)</code> for any <code>_STANDARD_SLOTS</code> member (~L134)	<code>select</code> names <code>item_type +</code> <code>priority →</code> 400 on drop → every <code>/schedule/*</code> route fails	Blocker
<code>routes/fields.py</code> <code>get_field_values()</code>	<code>select: slot + slot: "not.is.null"</code> filter when the mapped field's slot ∈ demoted set (~L133-148)	both the select column and the filter column become unknown → 400 on drop	Blocker
<code>routes/fields.py</code> <code>get_asset_combinations()</code>	<code>select_cols</code> built from <code>_STANDARD_SLOTS</code> membership names demoted columns (~L208-209); <code>_val()</code> reads <code>row.get(slot)</code> for any standard slot (~L224)	demoted columns land in <code>select</code> → 400 on drop	Blocker

Status: all three fixed in the working tree as of the Phase A-D + Step 1 changeset (see §2 Step 1). Each demoted reader now reads `meta["__slots"]` first; `scheduler.py` drops its explicit `select` (fetch-all, like `assets.py`) and `fields.py` routes demoted slots through `__slots` instead of a column select/filter.

Readers that are **safe** (verified, do not need pre-drop fixes): - `routes/assets.py`, both `_build_asset_response` feeder queries use **no explicit select**, so PostgREST returns all surviving columns; dropped keys are simply absent and `row.get( ... )` returns `None`. Fallbacks become dead code → Phase E cleanup only. - `routes/numbersbot.py`, `routes/reviews.py`, already migrated to `meta["__slots"]` (Phase B). - `routes/schedule.py` `_build_asset_description()`, reads the camelCase **output** dict (`itemType / team / priority`), not a raw row; correct automatically once `scheduler.py` is fixed.

Migration-file traps (both files are NOT safe to `supabase db push` blindly)

- `20260528000003_asset_slots_tier2.sql`: Part A (the backfill `UPDATE` and the index builds) is **entirely commented out**; only Part B (the `DROP COLUMN`s) is live SQL. A naive `db push` would **drop `item_type / team / status` with no backfill** → those values never reach `__slots` → permanent loss from the UI. Part A must be run by hand first.
- `20260528000002_asset_slots_tier1.sql`: backfill + drop are live in one transaction; the verify queries are commented. A `db push` backfills and drops atomically with **no human verify gate**. Lower risk (backfill does run) but you lose the checkpoint.

Treat both files as **manual procedures**, not push-and-forget migrations.

1. Preconditions

- ☐ Confirm current prod is running pre-demotion code (reads columns directly). If Phase A-D was already partially deployed, stop and reconcile state before continuing.
- ☐ Take/verify a recent DB backup or snapshot (column drops in Tier 2 Part B are the point of no return for the column data; `__slots` must be confirmed populated first).
- ☐ `supabase` CLI linked to the **prod** project, or have dashboard SQL access. (Dev project should go through this whole runbook first as a rehearsal.)
- ☐ Know how to trigger a full (non-delta) sync per studio, the writer only writes `__slots` on records it re-normalizes.

2. Ordered procedure

STATUS (2026-05-28): Step 1 is **DONE**, Phases A-D + the three missed readers (`scheduler.py`, `fields.py` ×2) are committed and pushed on `dev` (commit `6821d06`). The full sequence is **rehearsed clean on the dev DB** (see §3a). **Prod work resumes at Step 2**. Exact copy-pasteable prod commands (validated on dev) are in §5.

Step 1, Complete the code change set (the working tree is currently incomplete)

The uncommitted tree has Phases A-D for `normalizer.py`, `writer.py`, `assets.py`, `numbersbot.py`, `reviews.py`. **Add the two missed readers before committing:**

lib/scheduler.py 1. In the `build_schedule()` PostgREST call (~L108), remove `item_type` and `priority` from the `select` string. Keep `meta` (already selected). New select: `"name,product,project_date,meta,canonical_asset_id"` 2. Replace the direct reads with `__slots` -first-then-column-fallback (fallback safe while columns still exist; becomes dead in Phase E): - `asset_item_name = asset_row.get("item_type") → (asset_row.get("meta") or {}).get("__slots", {})`.get("item_type") or `asset_row.get("item_type")` - `strategic_priority = asset_row.get("priority") → same pattern for priority` 3. In the `f()` resolver, demoted slots must not read `asset_row.get(slot)`. Either drop the five demoted slots from `_STANDARD_SLOTS` and let them resolve from `__slots`, or special-case: `if slot in _DEMOTED: return (asset_row.get("meta") or {}).get("__slots", {}).get(slot)`. Verify against the estimate-variable resolution path before choosing.

routes/fields.py, two endpoints: 1. `get_field_values()`: for a demoted slot, the value now lives in `meta["__slots"]` `[slot]`, not a column. Replace the `select: slot + slot: not.is.null` branch with a fetch-all (no explicit select) that reads `(row["meta"]["__slots"] or {}).get(slot) or row.get(slot)`, fetch-all keeps the pre-migration column fallback without 400-ing post-drop. Keep the column path only for non-demoted standard slots (`name`, `product`, `project_date`, `asset_number`). 2. `get_asset_combinations()`: exclude demoted slots from the dynamically-built `select_cols` (it must never

name a dropped column); have `_val()` read demoted slots from `meta["__slots"]`. 3. Add a `_DEMOTED_SLOTS` frozenset so both endpoints treat the five demoted slots as `__slots` reads, not column reads.

Note: `lib/scheduler.py` and `routes/fields.py` each carry their own `_STANDARD_SLOTS` frozenset still listing the demoted columns, both must be reconciled. (There is no shared constant; this duplication is itself worth a follow-up.)

- ☐ All five readers (`assets.py` ✓ already done, `numbersbot.py` ✓, `reviews.py` ✓, `scheduler.py`, `fields.py`) read `__slots`-first.
- ☐ Grep sweep returns no remaining `select` string or PostgREST filter naming `dev_name / item_type / priority / team / status` against `replicated_assets`. (`status` on `replicated_work` /sync/invite/job is unrelated, do not touch.)

Step 2, Commit + deploy the code (columns still present)

- ☐ Commit Phases A-D + the scheduler/fields fixes as one focused changeset. Do **not** include the column-drop migrations in a way that auto-runs on deploy.
- ☐ Deploy to prod. Now: sync writes `__slots`; writer no longer writes the five columns; all readers prefer `__slots` and fall back to the still-present columns for not-yet-resynced rows. Nothing is broken because columns still exist and old rows still have their values.

Step 3, Full sync per studio (optional but recommended)

- ☐ Trigger a full sync for each active studio so live records populate `__slots`. This shrinks the set the migration backfill has to cover and validates the write path in prod before any drop.

Step 4, Tier 1 migration: `dev_name`, `priority` (run manually, with the verify gate)

Do **not** rely on a blind `db push` (it would skip the commented verify gate). Run in three explicit statements:

- Run the backfill `UPDATE` (the live SQL at the top of `20260528000002`).
 - Run the verify queries (currently commented in the file), **all must return 0**: `sql SELECT count(*) FROM replicated_assets WHERE dev_name IS NOT NULL AND NOT (meta->'__slots' ? 'dev_name');` `SELECT count(*) FROM replicated_assets WHERE priority IS NOT NULL AND NOT (meta->'__slots' ? 'priority');`
 - Only if both are 0: run the `DROP INDEX` + two `DROP COLUMN` statements.
- ☐ After drop, smoke-test: asset list/detail render `dev_name` + `priority` (now from `__slots`); `/schedule/*` runs (this is where the unfixed `scheduler.py` would have 400'd on `priority`).

Step 5, ### Step 5: Tier 2 Part A: backfill + indexes (run manually, Part A is commented in the file)

Tier 2's backfill and indexes are commented out in `20260528000003`. **Uncomment/execute them by hand.** None of Part A runs inside a transaction.

- Run the batched backfill loop. Start `:last_id = '00000000-0000-0000-0000-000000000000'`, advance to the MAX id of each batch, repeat until 0 rows affected. Idempotent; safe to restart from the zero UUID.
 - Build the three `CREATE INDEX CONCURRENTLY` indexes (`item_type`, `team`, `status`), indexes (`item_type`, `team`, `status`), each as a standalone statement, **not** in a transaction. A failed build leaves an INVALID index, `DROP INDEX` before retry.
 - Verify all three return 0: `sql SELECT count(*) FROM replicated_assets WHERE item_type IS NOT NULL AND NOT (meta->'__slots' ? 'item_type');` `SELECT count(*) FROM replicated_assets WHERE team IS NOT NULL AND NOT (meta->'__slots' ? 'team');` `SELECT count(*) FROM replicated_assets WHERE status IS NOT NULL AND NOT (meta->'__slots' ? 'status');`
 - Confirm the three CONCURRENTLY indexes are VALID: `sql SELECT indexname FROM pg_indexes WHERE tablename='replicated_assets' AND indexname LIKE 'replicated_assets_slot_%';`
- ☐ All three verify queries = 0. All three indexes VALID. (`team` is expected NULL on all rows, is expected NULL on all rows, it was never written by the writer, so its verify is trivially 0; that's fine.)

Step 6, ### Step 6: Tier 2 Part B: drop columns (transactional, the point of no return)

Run only after Step 5 passes. This is the live `begin; ... commit;` block in the Tier 2 file (the only part that runs on a blind push, which is exactly why a blind push is dangerous).

- ☐ Run Part B (`DROP INDEX` ×2 + `DROP COLUMN item_type, team, status`).
- ☐ Full smoke test: asset list/detail, NumberBot context, reviews meta panel, `/schedule/*` (**scheduler.py select no longer names item_type**), fields `get_field_values` for an item_type/status/priority-mapped field (fields.py no longer selects the column).

Step 7, Phase E: remove dead fallbacks (cosmetic, separate PR)

Now that columns are gone, the `or row.get( ... )` fallbacks are dead code. Remove them per the `project_todo_slot_demotion_phase_e` memory: - `routes/assets.py _build_asset_response()`, the five `_slots.get(x)` or `row.get(x)` lines → `_slots.get(x)` or `None`; and the status injection `... or row.get("status")` → drop the column fallback. - `lib/scheduler.py`, `routes/fields.py`, remove the column fallbacks added in Step 1 (the `or asset_row.get( ... )` tails). - Remove the "Slot demotion Phase E pending" entry from `CLAUDE.md` Known Debt. - Update memory: mark `project_asset_slot_demotion` and `project_todo_slot_demotion_phase_e` DONE.

3. One-line sequence

`fix scheduler.py + fields.py` → `commit + deploy all reader/writer code` → `full sync` → `Tier1: backfill` → `verify=0` → `drop` → `Tier2 Part A: backfill` → `indexes` → `verify=0` → `Tier2 Part B: drop` → `smoke test /schedule + fields` → `Phase E dead-code removal`.

3a. Rehearsal log (dev project kwr'lqqnzcnpjqvesygxo , 2026-05-28)

Full sequence rehearsed end-to-end on dev via `supabase db query --linked`. Clean.

- Baseline: 247 assets; priority/item_type non-null on 238, status on 236, dev_name on 1, team on 0; **0** rows had `__slots` (pre-deploy state, exactly the column→`__slots` backfill path).
- Tier 1 backfill → verify `devname_missing=0`, `priority_missing=0` → dropped `dev_name`, `priority`. ✓
- Tier 2 Part A backfill → verify `it=0`, `team=0`, `status=0` → 3× `CREATE INDEX CONCURRENTLY` all built **VALID** → Part B dropped `item_type`, `team`, `status`. ✓
- All five columns confirmed gone; `__slots` carries the values (null-valued keys for dev_name/team, as `jsonb_build_object` produces, benign).
- **PostgREST validation against post-drop schema:** no-`select` query → **200** (dropped cols absent, `__slots` present); `select=item_type` → **400** column does not exist; `item_type=not.is.null` filter → **400**. Confirms the fixed readers work and the old readers would have broken.

Operational findings for the prod run: - `supabase db query --linked` executes `CREATE INDEX CONCURRENTLY` successfully, it does **not** force a transaction wrapper. So the whole sequence (incl. Tier 2 Part A) can be driven through `supabase db query --linked` with explicit per-statement calls; a separate `psql` is **not** required. (Still run statement-by-statement with the verify gates, do not `supabase db push` the files, which would skip the gates / run only the live SQL.) - Multi-statement DDL (`DROP INDEX ...; ALTER TABLE ... DROP COLUMN ...;`) works in a single `db query` call. - For the 247-row dev table the Tier 2 backfill ran as a single `UPDATE` (no cursor batching). Prod must still use the batched cursor loop if `replicated_assets` is large.

4. Rollback notes

- Steps 1-3 (code) roll back by redeploying prior code; `__slots` writes are additive and harmless to old code (which ignores `__slots`).
- Step 4/6 column drops are **not** reversible without restoring column data from `__slots`. If a drop must be undone: `ADD COLUMN`, then `UPDATE ... SET col = meta->'__slots'->>'col'`. Re-add the dropped indexes. Treat as incident recovery, not routine.

- The Tier 2 CONCURRENTLY indexes (Step 5) are droppable any time with no data impact.

5. Prod execution, exact commands (validated on dev)

These are the literal commands the dev rehearsal ran, ready for prod. Run each block **separately**, confirm each verify returns 0 **before** the matching drop. Do **not** `supabase db push` the migration files, that skips the verify gates and (for Tier 2) runs only the drops with no backfill.

⚠ **Re-link the CLI to the PROD project first.** Prod is a *separate* Supabase project from dev (`kwrlqqnzcnpjqvesygxo` is dev). Confirm the target before any drop: `bash supabase link --project-ref <PROD_PROJECT_REF>` `supabase db query --linked -o csv "select current_database(), inet_server_addr();" # sanity check you're on prod` Every command below uses `--linked`; it hits whatever project is currently linked. Getting this wrong drops prod columns against the wrong DB.

Step 2-3: deploy commit `6821d06` (or its merge into the prod branch) → trigger a full sync per studio.

Step 4, Tier 1 (`dev_name` , `priority`):

```
# 4.1 backfill
supabase db query --linked -o csv "UPDATE replicated_assets SET meta = jsonb_set(coalesce(meta, '{}')::jsonb), {'_
# 4.2 verify - BOTH must be 0
supabase db query --linked -o csv "select (select count(*) from replicated_assets where dev_name is not null and
# 4.3 drop - only if 4.2 is 0,0
supabase db query --linked -o csv "DROP INDEX IF EXISTS replicated_assets_owner_type_owner_id_priority_idx; ALTER
```

Step 5, Tier 2 Part A (`item_type` , `team` , `status`): backfill + indexes

```
# 5.1 backfill. On dev (247 rows) this single UPDATE was fine. If prod replicated_assets is
# large, use the batched cursor loop from the migration file instead (LIMIT 5000 by id).
supabase db query --linked -o csv "UPDATE replicated_assets SET meta = jsonb_set(coalesce(meta, '{}')::jsonb), {'_
# 5.2 verify - ALL THREE must be 0
supabase db query --linked -o csv "select (select count(*) from replicated_assets where item_type is not null and
# 5.3 indexes - db query --linked runs CONCURRENTLY fine (no txn wrapper); run each separately
supabase db query --linked -o csv "CREATE INDEX CONCURRENTLY replicated_assets_slot_item_type_idx ON replicated_a
supabase db query --linked -o csv "CREATE INDEX CONCURRENTLY replicated_assets_slot_team_idx ON replicated_assets
supabase db query --linked -o csv "CREATE INDEX CONCURRENTLY replicated_assets_slot_status_idx ON replicated_asse
# 5.4 confirm all three indexes VALID (indisvalid = true)
supabase db query --linked -o csv "select c.relname, i.indisvalid from pg_class c join pg_index i on i.indexrelid
```

Step 6, Step 6, Tier 2 Part B (drops, point of no return):**

```
# only after 5.2 = 0,0,0 and 5.4 all true
supabase db query --linked -o csv "DROP INDEX IF EXISTS replicated_assets_owner_type_owner_id_item_type_idx; DROP
# confirm all five gone
supabase db query --linked -o csv "select coalesce(string_agg(column_name, ','), '(none remain)') from informatio
```

Post-drop smoke test (against prod): asset list/detail, NumberBot context, reviews meta panel, `/schedule/*` (build_schedule no longer selects item_type/priority), and a fields `get_field_values` / `get_asset_combinations` call for an item_type/status/priority-mapped field.

Step 7, Phase E: once prod is verified, remove the dead `or row.get( ... )` / `or asset_row.get( ... )` fallbacks (`assets.py` , `scheduler.py` , `fields.py`), drop the CLAUDE.md Known-Debt entry, and mark the two slot-demotion memories DONE.